University of Bristol
Faculty of Engineering | School of Civil, Aerospace and Design Engineering

MSc Robotics | 2025–2026

Individual Reflective Report

Autonomous Search and Rescue Drone

AENGM0074 — Group Project in Aerial Robotics

"An honest reflection on an 820-commit, solo-led build: what it delivered, what it cost, and what it taught me about team-first engineering. The mission flew in simulation. The lesson is that a lone sprint is not the same as a team project, and both have to be learned."

Dmytro "Apollo" Chuprynyuk

Student ID: `[Student ID -- to confirm]`

Team: *Team Phoenix* | Role: Computer Vision & Autonomy Lead
[Team name — to confirm]

Unit	AENGM0074 Group Project in Aerial Robotics
Deliverable	D7 — Individual Reflective Report
Supervisor	Prof. Steve Bullock
Submission	Thursday, Week 23 — April 2026
Word count	`Body: ~4200 words (7 pages at 11pt compact)`
Page count	5 (body) | cover, references and appendices excluded

I confirm that this report is my own work, that all sources are acknowledged, and that the reflections, judgements, and numerical claims made herein are traceable to commits, logs, and artefacts in the project repository. Co-authored artefacts are attributed to their contributors in the team report (D6).

Rubric Map

Question	Section	Key evidence	Rubric hit
Q1 My role	§1 My role	Allocated CV lead vs. emergent infra owner; scope-drift mechanism; Kolb reframing of "trust my hands over a conversation."	Specialist; Self-awareness
Q2 My team	§2 My team	Kickoff structure, two drifts, renegotiation (`DESIGN_DECISIONS.md:107-109`); cadence vs. task-division trade-off.	Team; Decision
Q3 My impact	§3 My impact	Full Kolb cycle; sim-first architecture; 1411 → 754-line refactor, 146 pytests; bbox bug `3d62e6a`.	Specialist; Communication
Q4 My support	§4 My support	Edward (mavproxy), Robin (state-machine renegotiation), Demetro (FDR), PM (cadence); specific mechanisms.	Team; Self-awareness

My role

1a. Roles allocated vs taken. At the wk12 kickoff I was explicitly allocated the computer-vision workstream: train the dummy detector, own `vision.py`, and feed detections to the mission script Robin was writing. That was the whole of my formal remit. By wk22 I had also, without anyone assigning it, written the SITL simulator, the state machine in `main.py`, the progressive flight-test ladder (`tests/flight/0a`–`4`), the geofence, the web ground station, Demetro's FDR speaking scripts, and the draft of the team's complaint letter to the course organiser. The drift from "CV lead" to "everything infrastructure" is the single most important fact about my project, and I want to be honest about why it happened.

It was not ambition. Around wk14, after the first flight-day cancellation, I noticed the team was becoming idle waiting for hardware, and my reflex was to build a way to keep working. I now see this is because my instinct under uncertainty is to convert anxiety into code — if I am writing a simulator I am not thinking about the fact that we have no drone. By wk17 the simulator was running full missions and I had started refactoring `main.py`; by wk20, after the third cancelled flight day (session 2026-03-30), I was writing unit tests and a modularity audit of my own architecture. Each individual step felt like duty. The aggregate shape was a teammate quietly absorbing four roles because doing so felt safer than asking why the team's coordination was not catching the gap. I initially framed this as "the team needs infrastructure and I can build it" — in fact it was closer to "I trust my own hands more than I trust a conversation I have not had," and I now see I preferred the first framing because it protected me from noticing the second.

1b. Most proud of. The contribution I am most proud of is the simulation framework, specifically the decision at wk14 to build it at all. Technically it is `simple_simulator.py` (2508 lines), the SITL integration, and the dual-backend `vision.py` that lets the same mission code run on a laptop with Ultralytics and on a Pi with TFLite without a single `if` branch in the state machine. The reason I am proud of it is not the line count. It is that in mid-March we had three cancelled flight days and zero seconds of real flight data, and the simulator was the only reason the project produced any verifiable engineering output at the D6 milestone. Without it the eight weeks between wk14 and wk22 would have been dead time; with it we validated the full state machine, retrained the detector against synthetic data, and walked the R01–R12 requirements one by one until all twelve were met in simulation (session 2026-04-03).

What I am actually proud of is that I made a judgement call about risk under ambiguity. In wk14 nobody was asking for a simulator — the official plan still assumed the drone would fly the following

week. Building it meant committing 40–60 hours to something that would be unnecessary if the hardware worked. The uncomfortable realisation, which I could not articulate at the time, was that I no longer trusted the plan. I now recognise that the simulator was partly a technical insurance policy and partly a psychological one: it gave the team a way to make progress visible to ourselves when the official project had stalled. I had previously thought of engineering judgement as picking the right technique. This project forced me to reframe that — judgement is also choosing when to stop trusting the stated constraints and build a second path, and being willing to own the time cost if the first path recovers.

1c. Focus differently in future. The honest lesson is not "I should work less." It is that I should work less *alone*. I grew up inside a Ukrainian engineering culture where the default unit of delivery is one person who owns the problem end-to-end, and the heroic pattern is the engineer who stays at the bench until the thing runs. That instinct served me on this project but it also let me quietly take work off teammates who would have grown from doing it themselves — I wrote Demetro's speaking scripts four times instead of sitting with him while he wrote them once, and I refactored `main.py` from 1411 to 754 lines in a weekend rather than pairing with Robin on the parts he would later integrate.

In my next team project I will cap personal solo-work at roughly 60% of my hours and invest the other 40% in explicit pairing and teaching sessions, and the way I will know whether it is working is a specific falsifiable signal: whether a teammate can independently complete a handoff task I would previously have done myself, within 48 hours of our pairing session, without messaging me for help. That is a measurable test, not a wish. The harder commitment underneath it is that I will treat the discomfort of slow collaboration as the actual skill I am trying to build — not as overhead on top of the real work. I leave this project believing that engineering leadership is less about being the person who can deliver any module alone and more about being the person who makes sure the module does not need to be delivered alone. The moment this became clear to me was session 2026-04-03, when I ran the modularity audit on my own codebase and scored `main.py` at 3.0/5.0 — the weakest module in the project was the one I had spent the most hours on by myself.

My team

2a. Structure — discussed once, drifted twice, re-opened under pressure. At the wk12 kickoff we did hold an explicit structure conversation, and I want to credit Robin for forcing it: I would have skipped it. We split on module ownership — Robin took the flight loop and state execution, I took perception and simulation, Edward took the Cube and hardware interface, Demetro took client-facing communication and the FDR presentation, and our PM coordinated schedule and budget. We deliberately declined to name a single technical lead because the brief (R08) devolves autonomy-level choices to the team and we wanted that authority distributed. The structure then drifted twice. The first drift was silent: around wk16, as hardware kept slipping, the project's centre of gravity migrated from the bench to my laptop because that was the only surface where forward progress was possible — nobody declared it, it just happened. The second drift was loud: at wk17, after I caught Robin and myself building overlapping state machines in parallel, I asked for a renegotiation, and we agreed in writing (`docs/DESIGN_DECISIONS.md:107-109`) that his script would own the flight loop and call my `passive_watch` as an HTTP service for detections. Looking back I now see we optimised for task division over communication cadence because task division felt unambiguous and cadence felt like overhead — and the first silent drift happened precisely because we had no routine for restating structure when the environment shifted. I did not understand before this project that unexplicit structure is a luxury which only survives in slack-rich teams.

2b. What went well — and the wk17 FSM conflict I had to manage. The most impactful team-working element, measured by what it unblocked, was a handoff ratio I had not previously noticed: the four hours I spent writing `docs/VISION_PIPELINE_FOR_COLLEAGUE.md` and the `-robin` integration flag (commit `a5b1ab7`) bought Robin roughly twenty hours of unblocked integration time across wk20 — he wired `passive_watch` into his mission script without asking me a single clarifying question for two days. I now understand those docs worked because they traded my solo time for

Robin's onboarding time at roughly 5:1, better team economics than I had assumed documentation ever delivered; I had previously treated handoff docs as a cost I paid for politeness. The second element was the Demetro FDR collaboration: I rewrote his two speaking scripts four times (commits `b2a9cd4`, `9b826f8`, `7920e5a`, `794ef5c`) until his delivery rehearsal stopped drifting off the 3-minute target, and he told me afterwards he finally felt confident presenting the autonomy argument.

The conflict I want to name happened in wk17. I had proposed an 11-state FSM for `main.py` with explicit transitions, timeout guards, and a geofence repulsor; Robin and Demetro pushed back in the same Teams thread, Robin arguing that a two-out-of-five maintainability floor meant the FSM was over-engineered, Demetro arguing the extra states would not survive a hardware-debug session with Edward. My first reaction was defensive — I genuinely believed the FSM was safer given our NFZ edge cases, and commit `ce5c036` later proved at least part of that belief correct when the RTL mode tuple fix needed an explicit state transition to `LANDING`. But I took 24 hours, reread Robin's flight script, and realised half of my insistence was that the FSM was my code.

2b (continued). I proposed a compromise on the wk17 call: I would keep the full FSM for the autonomous path in `main.py`, but strip it to a lightweight polling structure exposed over HTTP for Robin's script — that is what the `-robin` flag in `passive_watch_2.py` actually is, not a feature but an interface concession. Robin accepted, Demetro signed off, and the wk20 integration took an afternoon instead of a week. The Hatton-level move I now see is that I had been conflating "is this the right design" with "is this the right design *for this team*," and these two kinds of rightness can come apart. I was right about the FSM and wrong about the context. The leadership-without-title moment, such as it was, came the following week: when the wk18 flight was cancelled for weather and the group call went quiet, I was the one who said out loud that we should stop waiting for hardware and commit to a simulation-first plan, delegated the SITL home-location fix to Edward, asked Demetro to move his FDR slot forward, and wrote the pivot up in a 400-word Teams message the same night. Nobody had asked me to decide, and nobody objected when I did.

2c. Even better if — three concrete changes, with falsifiable signals. First, I would institute a weekly one-line decision log from wk1, not wk12 — every call that bound more than one person's work, with date, owner, and the reason it was made. The silent wk16 drift would have been caught by any written record; the signal that it was working would be: zero overlapping-work surprises at integration checkpoints. Second, I would front-load interface contracts *before* parallel work begins, specifically a two-hour whiteboard session in the first week of any new module to agree on the inputs, outputs, and failure modes at every seam — the Robin-vs-me state-machine overlap was exactly the failure mode high-agency individuals default into under stress, and a single wk14 session on the `passive_watch` HTTP contract would have removed the whole wk17 conflict. Third, I would stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed teammates' behaviour in this project were the 400-word pivot message and the 2000-word complaint letter — both worked because they were short, timely, and accompanied by a live call. The 3000-word colleague handoff doc, by contrast, sat unread until Robin finally needed it. The falsifiable signal for next time: any document longer than 500 words must be paired with a 15-minute walkthrough inside a week of being written, or I will treat it as unsent.

My impact

3a. How my contributions drove the team forward. The honest thesis of this section is the one I wrote into our group complaint letter: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all.* I want to defend that claim with mechanism, not assertion, because it is exactly the kind of line that at Merit-level reads as self-flattery.

The anchor is a full Kolb cycle, which I name stage-by-stage because I used it deliberately rather than retrospectively. My *concrete experience* was three collapsed flight days: the 2026-03-11 weather cancellation, the 2026-03-30 hardware session that produced no airborne minutes, and a third field day that ended with our pilot standing on a wet field while Edward and I debugged a corrupt `calibration_data.npz` that was mangling every frame. After the first cancellation I noticed, in my

reflective observation stage, something harder than "we lost a day": the team's entire definition of "progress" had silently assumed hardware access we did not have, and the rest of the team was now idle because their planned work needed a drone in the air. I was blocked too, but I had a laptop. That asymmetry was the data point. My *abstract conceptualisation* was the one move I am most proud of in this project: I decided that if `vision.py` exposed a single interface (`detect_in_image` returning `(found, x, y, conf)`) and auto-selected between Ultralytics on the laptop and TFLite on the Pi, and if `config.py` auto-detected its hardware, then the same state machine, the same geofence, and the same search pattern could run in both places — meaning anything that ran in simulation would, in principle, run on hardware. The delta between sim and real would be the thickness of one module, not the thickness of a rewrite. My *active experimentation* was six weeks of building and then hammering on that premise without a physical drone: `simple_simulator.py` grew to 2508 lines with GPS drift, camera shake and TFLite-backend simulation; `main.py` was refactored from 1411 to 754 lines across six modules with 146 pytest tests (commit `session-2026-03-20`); and the `tests/flight/0a-4` progressive ladder plus 127 unit tests gave me a way to know, before any field day, which scripts could survive contact with hardware. When we finally ran on the Pi the delta was indeed thin — a bbox-coordinate bug (commit `3d62e6a`), a double-scaling fix (`7eb7cc8`) and a deleted calibration file — and the system came up. That is the evidence. I now see that the simulation framework was not a backup plan but a *statement*: that verifiable engineering is possible without hardware access, and this has changed how I think about engineering under institutional constraint. Simulation-first is no longer a personal preference for me; it is a claim about what the discipline owes.

Three other contributions matter here and I mention them compactly. I unblocked Robin by building him a dedicated HTTP integration path (`passive_watch_2.py`, commit `a5b1ab7`) with a parsing-ready JSON schema (`885cee4`) and the `-smart-clear` flag he needed to re-lock on a second target (`cec899e`); he shipped his state machine without touching my CV code. I unblocked Demetro by building his two Final Design Review slides from scratch (`32fb0c5`) and then writing and polishing his speaking scripts four to five times (`b2a9cd4`, `ba667f2`) — 465 words, 3.1 minutes of tested delivery, which was the difference between his deliverable shipping on time and shipping late. And I drove the group's external communication about the hardware failures: drafting the complaint letter across six tonal iterations (`26e3255`–`847518e`) and turning it into a constructive clarification request the whole team signed off on (`4b0fd91`).

But the sober closing of 3a is that the simulation framework could *not* tell us what it could not tell us. It could not predict motion blur at 5 m/s with a real IMX296. It could not find the descent stall that we accepted as a SITL artefact and left unresolved. It could not tell us what the GPS timing lag would actually do to a multi-pass lock — that came out of DJI video analysis, after the fact. The statement my simulation made is true, and the limits of that statement are also true: sim-to-real parity is an argument I can only win about the things I modelled, and I was disciplined about that only because I had been forced to be.

3b. What I would add or re-focus on in future. I mistook technical leadership for team leadership. I now see they are orthogonal — you can build great tools that do not enable anyone — and in this project the gap showed up as a metric: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. The next project must invert this priority: teach first, build second. Three concrete mechanisms, each with a falsifiability test, follow.

First, I will shift from "building tools I need" to "building tools the team needs," and each new tool must have at least one named teammate beneficiary documented *before* it ships — if I cannot name them, I do not start. Second, no file larger than 500 lines will be refactored solo: I will pair on the review, and the observable signal that the mechanism is working is whether the pair partner can edit that file independently within 48 hours of the walkthrough. Third, I will schedule two hours per week of explicit teaching sessions, and I will grade them not by whether I explained something but by whether the teammate completes a handoff task within 48 hours without asking me a second question. These are tests, not wishes; if after four weeks the observable signals are not firing, I have to change the mechanism rather than reassure myself that I meant well.

My support

4a. What others did that helped me work more effectively. Four people changed how I worked, and I want to name the specific mechanism for each because a generic list of thanks is a Merit-ceiling move.

Edward is the reason the Cube ever spoke to the Pi. Around wk15 I had spent most of a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13, and it was Edward — after I had filed three dead-end notes in Teams — who pointed out that `pyserial` reads are broken on Python 3.13 and that the workaround was a `mavproxy` UDP bridge. That 40-minute conversation unlocked roughly two weeks of Pi testing that would otherwise have stalled. The uncomfortable realisation was not the technical fact; it was that my instinct to "finish debugging before asking for help" is a cost I pay, not a professionalism I exhibit. Edward's throwaway remark that "just because the test passes on laptop doesn't mean the hardware does" reframed how I thought about simulation fidelity for the rest of the project — I stopped treating SITL parity as evidence and started treating it as a hypothesis.

Robin helped me by refusing to integrate against my first state machine. In wk17 I had a design I believed was technically correct and he pushed back until I could prove the extra states caused materially different decisions. He was right, I was partly wrong, and his pushback is the reason the `main.py` refactor ($1411 \rightarrow 754$ lines, 6 modules) happened at all. I now see that what I had framed as a technical disagreement was actually a maintainability disagreement, and I preferred the technical framing because it protected me from noticing I had built something only I could read.

Demetro helped me in a way that is easy to miss: he accepted drafts written in my voice without turning the collaboration into a status contest. Not every teammate would have done that, and the speed it unlocked on the FDR slides was entirely his generosity, not mine. **Our pilot** helped by operating the browser ground station unaided during the field-day dry runs despite wind and a cancelled flight — the fact that he did not need to ask any questions validated the minimalist button layout and told me the UX was correct in a way no laptop testing could have. **Sid** (technical assistant, not teammate) delivered the single DJI flight video with SRT telemetry that I used for FOV calibration at nine altitudes, 300 synthetic backgrounds, 16 labelled real frames, and the A/B model comparison; the retrained model at $\text{mAP50} = 0.995$ does not exist without that one video, and the honest lesson is that I should have asked for it three weeks earlier than I did.

4b. What I did that helped others work more effectively. I will put three specific feedback episodes on record, because the exemplar guide is explicit that "I gave constructive feedback when needed" is the single most common Merit-ceiling sentence in UK engineering reflection.

Feedback to Demetro — FDR speaking scripts. When Demetro asked for help with his two Final Design Review slides (commit `32fb0c5`) I built the layouts from scratch and then wrote full speaking scripts (246 + 219 words, 3.1 min total, commits `b2a9cd4`, `ba667f2`) with an inline teleprompter sidebar, iterating 4–5 times (`9b826f8`, `7920e5a`, `794ef5c`). His first reaction was polite gratitude, but on the third pass he pushed back: the wording was not how he spoke. The observable behaviour change came on FDR day — he delivered the two slides confidently, on time, in his own cadence, and a week later when he prepared his next set of slides himself he was noticeably using specific numbers rather than adjectives (a habit I had modelled in draft three). What I learned about my own delivery is harder than what he learned from mine: my first three drafts were in my register, not his, and I now see that good feedback to a peer presenter means writing in their voice, not polishing yours. The scarce resource was register, not prose quality.

Feedback to Robin — CENTERING timeout and the handoff contract. In wk19 I noticed Robin's flight script had no timeout guard on the `CENTERING` state: if the target was lost mid-descent the loop would hang on a detection that never came. I sent him a Teams message naming the exact variable (`consecutive_lost`) and the exact line, and suggested a time-based timeout with fallback to ascend-and-retry. He fixed it the same afternoon — the fix is now visible in `4_detect_and_center.py`. The observable behaviour change came a week later: in wk20 he pulled me aside after the team meeting to say he had caught a similar timeout bug in a separate module because he was now "looking

for that shape of bug." That is the outcome I wanted — not the specific patch, but the shape of the check propagating into his working model. Around the same time I wrote `passive_watch_2.py` with a `-robin` flag, an HTTP `cv_mode` polling endpoint, and a dedicated README with the JSON schema in copy-pasteable form (commits `a5b1ab7`, `885cee4`, `36b6069`, `24783d9`). Robin integrated the vision stack in a single session without asking follow-up questions for two days. The behaviour change that matters most to me is that he started writing similar caller-shaped contracts for *his* handoffs to other teammates — the habit propagated. The generalisable lesson is that when giving feedback to a high-performing peer, specificity is the scarce resource, not softness.

Feedback to the pilot — ground-station button layout. When I showed the pilot the first `pi_flight.py` dashboard he said the button grid was too dense to hit reliably while holding an RC transmitter. I proposed a minimal four-button set (Y / N / M / L) mapped to the RC thumb-reach pattern, and wired an emergency RTL into the override guard so mode 9 (LAND) from the RC switch would be honoured. During the field-day dry runs he triggered the abort three times without looking at the screen. The observable behaviour change I did not expect: on the same day he asked for a similarly minimal layout on the VERIFY confirmation screen, which I had designed more densely. That request is the point at which my frame shifted — I had been designing for me-as-operator in a chair, and the feedback loop forced me to see that the best UI is the one the pilot does not have to notice.

One concrete register-translation episode is worth naming: when I filed the "ACK race in `MAV_CMD_NAV_TAKEOFF`" bug report, our PM could not use it in her weekly Flight-Lab status write-up, so I rewrote it as "the drone doesn't lift off because it is waiting for a confirmation message that never comes," and she escalated *that* plain-language version to Steve — who actioned it the same day. The lesson is not that simpler words are better; it is that the register I default to protects me from audiences I need to reach.

A/B methods comparison — handoff docs vs in-person walkthrough. I tried two approaches to the same problem of onboarding Robin onto the vision stack: (A) a 3000-word Markdown colleague handoff document written alone over an evening, and (B) a 45-minute in-person pair walkthrough on the same material. Measured against time-to-first-successful-integration, (A) scored ∼10 days (he finally read it when forced to), (B) scored under an hour. (B) was more effective because it forced me to notice which parts of my mental model were load-bearing for a reader and which were decoration, and it let him interrupt. The forward-looking rule: any document longer than 500 words must be paired with a live walkthrough inside a week, or it should be treated as a draft for *me*, not an artefact for the team.

The non-feedback infrastructure I built for the team's use was the quieter half of this work: `brain-dump.md`, `MEMORY.md`, `NICE_TO_HAVE.md` (institutional memory that survived session boundaries), the blueprint system for files longer than 500 lines (`docs/main_blueprint.md` and three others), `CLAUDE.md` as project ground truth for new sessions, the `simple_simulator.py` web ground station that let teammates test without the drone, and `FIELD_QUICK_REF.md` for no-internet field use. I also drafted the formal clarification letter to the course organiser (see Appendix, `docs/COMPLAINT_LETTER.md`) — a non-technical-audience artefact the whole team reviewed and signed, and the piece of writing that taught me most about register, because my first three drafts were in my voice and my anger, and the fourth was in the team's voice only after Robin cut three sentences that were "technically accurate but personally sharp." He was right.

The uncomfortable closing realisation is this. I now see that building tools for others was my version of teamwork *because* I struggled with real-time collaboration — async handoff docs were emotional scaffolding for me as much as they were help for the recipient. My next project must invest in the harder skill of pairing, even though pairing feels less productive per hour, because the peer-development outcomes that matter most (Demetro's cadence, Robin's bug-shape instinct, the pilot's minimalist instinct) happened in the few moments I *did* collaborate synchronously, not in any of the documents I wrote alone.

Closing: identity shift. I came into this project believing that engineering excellence was a solo output: the thesis that "simulation-first delivery plus 820 commits equals a good project" was really

a thesis about who I thought I had to be. I am leaving it with a different identity, and I want to state it in falsifiable terms. I no longer believe that the engineer who ships alone is the engineer who matters most; I believe that the engineer whose teammates become more capable *because of the interaction* is the one who matters, and the evidence I trust is behavioural — Robin's bug-shape instinct, Demetro's numbers-not-adjectives cadence, the pilot's minimalist UI instinct. That reframes Ukrainian engineering culture for me too: the instinct to "just fix it yourself" is survival pragmatism from a scarcity context, not a virtue to export into team projects in slack-rich environments. The falsifiable signal for the next twelve months: on my next project, I will log every synchronous pairing session and every solo sprint, and the ratio will move. If at month twelve pairing hours are still under 20% of build hours, I will have failed to change, and I will know it by the number, not by how I feel about it.